Fantasy Football Manager

Complete System Guide and Technical Documentation

Academic Technical Report

Application Version 0.1.0

Documented from the final project snapshot

September 1, 2026

Abstract

Fantasy Football Manager is a local decision-support system that combines a Python data and analytics engine with a JavaFX desktop interface. The system integrates Sleeper player metadata, FantasyPros consensus average draft position, and CBS statistical projections; applies league-specific scoring; calculates replacement levels, Value Over Replacement, positional tiers, and market-value indicators; and supports live snake-draft management through a persistent draft state and local FastAPI service. This report documents the final application architecture, datasets, analytical methods, service endpoints, desktop workflow, operational requirements, limitations, and recommended hardening roadmap. The analysis finds that the project is a functional and interpretable personal draft assistant whose primary remaining needs concern runtime resilience, automatic recalculation after configuration changes, testing, backup behavior, and standalone packaging.

Keywords: fantasy football, Value Over Replacement, average draft position, draft analytics, FastAPI, JavaFX, decision-support systems

Table of Contents

1. Executive summary

Fantasy Football Manager is a local desktop draft assistant that combines a Python data and analytics engine with a JavaFX desktop interface. It gathers NFL player metadata, consensus average draft position (ADP), and statistical projections; converts those projections into league-specific fantasy points; estimates position-specific replacement levels; calculates Value Over Replacement (VOR); builds tiers and model-versus-market value labels; and presents the results in a live draft board.

The finished system is not merely a static ranking spreadsheet. It maintains a persistent draft state, understands snake-draft order, tracks every team's roster, removes drafted players from the available pool, calculates recommendations when the user is on the clock, and estimates whether a candidate is likely to survive until the user's next selection.

The application has four major layers:

- **Data acquisition and preparation** — Python scripts download Sleeper player data and CBS projections, then clean a manually supplied FantasyPros ADP file.
- **Analytics** — Python calculates custom fantasy points, replacement levels, VOR, tiers, ADP value, review flags, and draft recommendations.
- **Local service layer** — FastAPI exposes league settings, draft state, available players, recommendations, and draft actions as JSON over localhost.
- **Desktop interface** — JavaFX starts the Python backend, communicates with FastAPI, and gives the user a single-window draft-night experience.

All information and state remain local to the computer except for the external source downloads performed by the data pipeline.

2. Current snapshot at a glance

The supplied ZIP contains the following final datasets:

Dataset	Rows	Purpose
Sleeper fantasy-player CSV	4,384	Broad NFL player identity and status pool
Clean FantasyPros ADP	602	Market draft-cost information
Master player dataset	599	Matched draft pool plus all 32 defenses
Clean CBS projections	441	Parsed statistical projections
Master players with projections	599	Identity, ADP, projections, and custom points
Final player values	599	Complete draft-analysis dataset

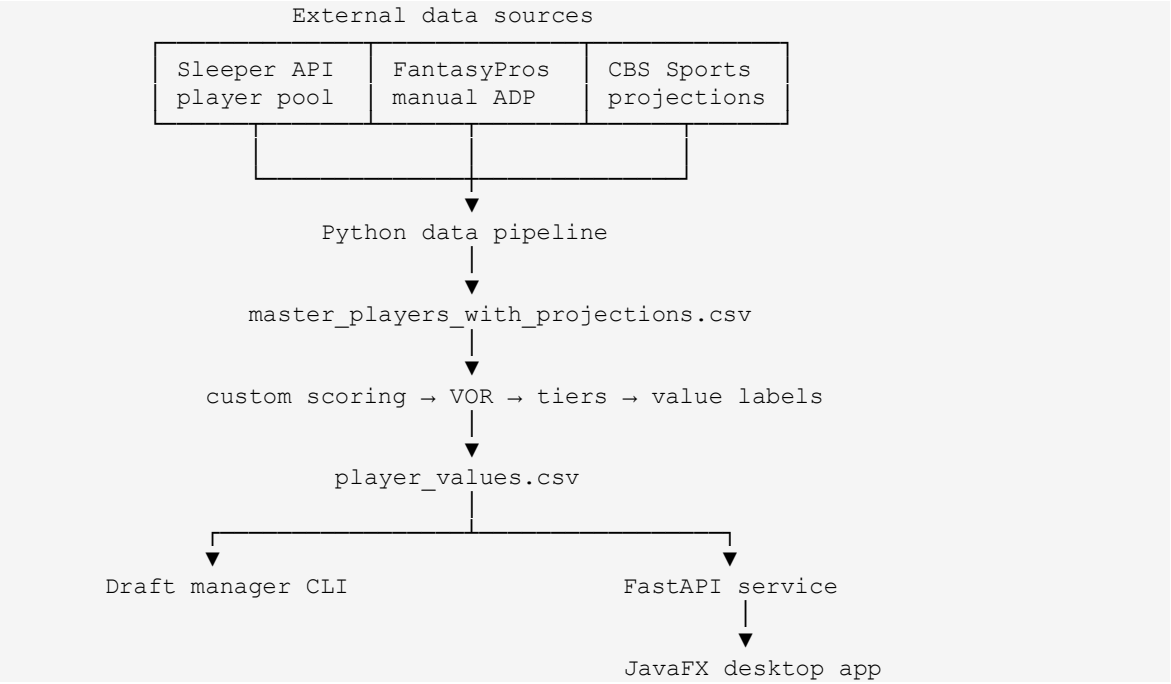
The final 599-player pool contains:

Position	Players
QB	88
RB	138
WR	196
TE	94
K	51
DEF	32

Within that pool, 395 players have projections, 329 offensive players have an overall model rank, 66 kickers/defenses have a special-teams rank, and 112 players are marked draft-relevant in the saved output.

An important snapshot detail is that `config.py` contains defaults of "Fantasy League" and 10 teams, but the included `python_engine/data/league_settings.json` overrides those defaults with **Hypebeast INC., 8 teams**. The saved draft state is likewise an eight-team, 14-round completed draft from slot 4. Runtime behavior follows the persisted JSON settings when that file exists.

3. Architecture



The CSV files are the analytical data store. `draft_state.json` is the transactional draft store. `league_settings.json` is the persistent runtime configuration store. FastAPI is intentionally thin: it translates JSON requests into calls to the Python draft engine and translates CSV/draft data into JSON responses. Java is the presentation layer rather than the analytical authority.

4. Repository structure

```
fantasy_football_manager/
├── datasets/
│   ├── adp.csv
│   ├── adp_clean.csv
│   ├── cbs_projections_raw.csv
│   └── cbs_projections.csv
├── java-app/
│   ├── pom.xml
│   └── src/main/java/com/hypebeast/ffm/
│       ├── ApiClient.java
│       ├── BackendService.java
│       ├── DraftPickRow.java
│       ├── FantasyFootballManagerApp.java
│       ├── LeagueSettingsDialog.java
│       ├── PlayerRow.java
│       └── RecommendationRow.java
├── python_engine/
│   ├── analytics/
```

```
│   │   ├── calculate_fantasy_points.py
│   │   └── calculate_vor.py
│   ├── api/main.py
│   ├── data/
│   │   ├── draft_state.json
│   │   ├── league_settings.json
│   │   ├── master_players.csv
│   │   ├── master_players_with_projections.csv
│   │   ├── player_values.csv
│   │   ├── players.csv
│   │   └── players.json
│   ├── draft/draft_manager.py
│   ├── scripts/
│   │   ├── build_master_dataset.py
│   │   ├── clean_adp.py
│   │   ├── clean_projections.py
│   │   ├── create_player_dataset.py
│   │   ├── download_players.py
│   │   ├── download_projections.py
│   │   ├── merge_projections.py
│   │   └── inspection utilities
│   ├── config.py
│   ├── paths.py
│   └── requirements.txt
├── refresh_data.command
├── run_pipeline.py
└── README.md
```

`paths.py` centralizes filesystem locations so scripts do not depend on the terminal's current directory. This is particularly important because the Java launcher starts Python as a child process.

5. League configuration

5.1 Defaults

`python_engine/config.py` defines defaults for league name, team count, roster construction, FLEX eligibility, bench assumptions, and scoring.

The saved roster format is:

- 1 QB
- 2 RB
- 2 WR
- 1 TE
- 1 FLEX

- 1 K
- 1 DEF
- 5 bench spots
- 1 IR spot

FLEX eligibility is RB, WR, and TE only. Quarterbacks are explicitly excluded.

5.2 Runtime overrides

At import time, `config.py` checks for `python_engine/data/league_settings.json`. If present, the JSON values replace the hard-coded defaults. This design lets the Java settings dialog persist league configuration without editing Python source code.

The settings dialog supports:

- League name
- Number of teams, from 2 through 32
- User's draft position
- Starter counts for QB, RB, WR, TE, FLEX, K, and DEF
- Bench and IR counts
- FLEX eligibility among RB, WR, and TE

The API refuses a team-count or draft-position change if picks already exist. Existing picks must first be undone or the draft reset. Other roster settings can be saved while picks exist.

5.3 Bench distribution

Replacement-level calculations require an assumption about how the league distributes bench spots. The model currently assumes an average five-player bench per team:

Position	Expected bench slots per team
QB	0.50
RB	2.25
WR	2.00
TE	0.25

These values are analytical assumptions, not hard roster limits. They influence replacement levels and therefore VOR. They are not currently editable through the Java settings dialog.

6. Scoring model

6.1 Offense

The custom point engine uses CBS projected statistics but applies local league scoring:

- Passing yards: 1 point per 25 yards
- Passing touchdowns: 4 points
- Interceptions: -1 point
- Rushing yards: 1 point per 10 yards
- Rushing touchdowns: 6 points
- Receptions: 1 point (full PPR)
- Receiving yards: 1 point per 10 yards
- Receiving touchdowns: 6 points
- Lost fumbles: -2 points

The configuration includes two-point conversions, but `calculate_fantasy_points.py` currently does not add a two-point conversion statistic because the cleaned CBS projection rows do not provide one.

For an offensive player, the basic formula is:

```
pass_yards / 25
+ pass_td × 4
+ interceptions × -1
+ rush_yards / 10
+ rush_td × 6
+ receptions × 1
+ rec_yards / 10
+ rec_td × 6
+ fumbles_lost × -2
```

6.2 Kickers

Kicker projections are calculated from distance-specific field-goal makes, total makes and attempts, and extra-point makes and attempts:

- Field goal, 0–19 yards: 3
- Field goal, 20–29 yards: 3
- Field goal, 30–39 yards: 3
- Field goal, 40–49 yards: 4
- Field goal, 50+ yards: 5
- Made extra point: 1
- Missed extra point: -1
- Missed field goal: -1

Misses are derived as `attempts - makes`, never below zero.

6.3 Defenses

Defense points include projected interceptions, safeties, sacks, fumble recoveries, forced fumbles, defensive touchdowns, and an estimated season-long points-allowed score.

The points-allowed component is approximate. CBS supplies average points allowed per game, not a projection of the exact scoring bracket in every game. The engine determines the bracket corresponding to the projected average and multiplies that bracket score by projected games. Because a real defense can move across brackets each week, this method is an estimate and every projected defense is marked `projection_is_estimate`.

Blocked kicks and some special-teams events exist in the configuration but are not added to the defense calculation because corresponding CBS projected fields are absent.

6.4 CBS versus league points

The merged dataset retains both:

`cbs_fantasy_points`

`projected_fantasy_points`

The first is CBS's published total. The second is the system's league-specific calculation. Keeping both is useful for sanity checks and for identifying scoring categories that CBS handles differently.

7. Data pipeline

`run_pipeline.py` is the one-command orchestration layer. It validates the required manual ADP file and runs eight steps in order.

Step 1: Download Sleeper players

`download_players.py` requests `https://api.sleeper.app/v1/players/nfl` and saves the raw response to `python_engine/data/players.json`.

Sleeper is the identity backbone because its records include player ID, full name, fantasy positions, NFL team, activity status, age, experience, depth-chart information, and injury status.

Step 2: Create the fantasy-player dataset

`create_player_dataset.py` converts the large Sleeper JSON object into a flat CSV. It retains QB, RB, WR, TE, K, and DEF records and writes

`python_engine/data/players.csv`.

Step 3: Clean FantasyPros ADP

FantasyPros is the only manual input. The user downloads the current CSV and saves it as `datasets/adp.csv`.

`clean_adp.py` validates expected columns, parses combined player/team/bye text, separates position from position rank, and writes `datasets/adp_clean.csv`. Consensus ADP is treated as the main market signal. Sleeper ADP is not required by the model.

Step 4: Build the master player dataset

`build_master_dataset.py` joins cleaned FantasyPros records to Sleeper identities. Matching proceeds conservatively:

- Normalized name + position + team
- Unique normalized name + position
- Unique normalized name only

The system records the match method for auditing. It also appends all 32 Sleeper defenses because overall FantasyPros ADP data may omit them. Output is `python_engine/data/master_players.csv`.

Step 5: Download CBS projections

`download_projections.py` requests CBS projection pages for QB, RB, WR, TE, K, and DST. Beautiful Soup extracts rows from the projection table, and the raw position-tagged data is saved to `datasets/cbs_projections_raw.csv`.

This is HTML scraping rather than a contracted API. A CBS page-layout change can break this step.

Step 6: Clean CBS projections

`clean_projections.py` translates position-specific CBS columns into a common 47-column schema. It:

- Converts em dashes and blank numeric cells safely
- Parses duplicated CBS player labels
- Skips unsupported fullbacks
- Maps CBS defense names to Sleeper team abbreviations
- Converts DST to the internal `DEF` position
- Corrects defense game counts when necessary
- Emits passing, rushing, receiving, kicking, and defensive statistics

Output is `datasets/cbs_projections.csv`.

Step 7: Merge projections and calculate custom points

`merge_projections.py` joins CBS rows to the master players using normalized identity keys and the same conservative fallback idea used earlier. On a successful match, it copies CBS statistics, stores CBS points, calculates league-specific projected points, computes points per game, and records the projection match method.

Unmatched players remain in the master pool with blank projections. This is intentional: free agents, deep reserves, and inactive players can still appear in the complete draft board.

Output is `python_engine/data/master_players_with_projections.csv`.

Step 8: Calculate draft values and tiers

`calculate_vor.py` converts projected points into league-contextual draft intelligence and writes `python_engine/data/player_values.csv`, the main dataset consumed by the draft manager and API.

Running the pipeline

From the project root with the virtual environment active:

```
python run_pipeline.py
```

The macOS helper `refresh_data.command` changes to the project directory and runs the same pipeline with `python3`.

8. VOR, scarcity, tiers, and market value

8.1 Expected rostered pool

The model first selects mandatory starters at each offensive position using league size and starter counts. It then fills all league FLEX spots with the highest-projected remaining RB/WR/TE players. Finally, it adds expected bench players according to `BENCH_DISTRIBUTION`.

This produces a league-specific expected rostered count at QB, RB, WR, and TE.

8.2 Replacement level

For each offensive position, replacement level is the lowest projected point total among the expected rostered players at that position.

```
VOR = projected fantasy points - position replacement points
```

This makes cross-position comparisons more useful than raw projected points. A quarterback can score more total points than a running back while providing less value above what will remain available at quarterback.

K and DEF are handled separately. Their replacement point is the projection of the final required starter at the position: K-N and DEF-N for an N-team league. Special-teams VOR is not mixed directly into the offensive overall model rank.

8.3 Tiers

Players with nonnegative VOR receive a position rank and tier. A new tier begins when the projection gap from the prior player is at least 5% of that position's replacement-level point total.

This is a cliff-based tier system: tiers communicate meaningful drops in expected output rather than arbitrary fixed group sizes.

8.4 Model rank versus ADP

Offensive players are ranked globally by VOR. The system then calculates:

```
ADP value = consensus ADP - model rank
```

A positive number means the model ranks the player earlier than the market. A negative number means the market is drafting the player earlier than the model.

Labels are assigned as follows:

ADP value	Label
15 or greater	Strong value
5 to 14.9	Value
-4.9 to 4.9	Fair value
-14.9 to -5	Reach
-15 or lower	Major reach

Players outside both the model's offensive draftable range and the overall market draft range are labeled deep players. Any draft-relevant player with an absolute ADP gap of at least 25 is marked for manual review, with a reason identifying whether the model or market is substantially higher.

The review flag is important. A large gap can indicate a genuine bargain, but it can also reveal stale projections, changed depth-chart roles, injuries, rookies with uncertain data, or a matching problem.

9. Draft engine

`python_engine/draft/draft_manager.py` is both a reusable business-logic module and a command-line application.

9.1 Persistent state

`draft_state.json` stores:

- Creation timestamp
- User's team name
- Draft slot
- Ordered team list
- Current overall pick
- Every recorded pick

Each pick stores overall pick number, round, pick within round, player ID, player name, position, NFL team, and drafting fantasy team.

9.2 Snake-draft order

Odd rounds move forward through the draft order; even rounds move backward. The engine derives round, pick within round, and team on the clock from the overall pick number and league size.

9.3 Draft actions

The engine can:

- Create or force-replace a draft
- Find a player by normalized name
- Record the current pick
- Assign a pick to the team on the clock or an explicitly supplied team
- Reject duplicate selections
- Undo the latest pick
- List available players
- Display all picks

- Display any team roster
- Produce recommendations
- Produce upcoming-pick targets and slide-watch candidates

9.4 CLI commands

```
python -m python_engine.draft.draft_manager new \
  --my-team "Jacob" --draft-slot 4 --force

python -m python_engine.draft.draft_manager draft "Player Name"
python -m python_engine.draft.draft_manager draft "Player Name" --team "Team 2"
python -m python_engine.draft.draft_manager undo
python -m python_engine.draft.draft_manager available --limit 20
python -m python_engine.draft.draft_manager picks
python -m python_engine.draft.draft_manager roster --team "Jacob"
python -m python_engine.draft.draft_manager recommend --limit 10
python -m python_engine.draft.draft_manager targets --limit 10
```

The Java application now covers most common draft-night actions, but the CLI remains valuable for troubleshooting or recovery.

10. Recommendation engine

Recommendations are generated only when the user's team is on the clock. When another team is drafting, the API reports the current team and the user's next pick but returns an empty recommendation list.

The recommendation score begins with VOR and applies modest context bonuses:

- Starter need: +15
- FLEX need: +7.5
- Player unlikely to survive to the next user pick: +10
- Borderline chance of surviving: +5

Return likelihood compares consensus ADP with the user's next pick using a four-pick window:

- ADP earlier than `next pick - 4`: Unlikely to return
- ADP through `next pick + 4`: Borderline

- Later ADP: Likely

Roster need is classified as STARTER, FLEX, or DEPTH. FLEX need considers how many eligible players exceed the mandatory RB/WR/TE starter requirements.

K and DEF are normally withheld from the main recommendations. They are promoted when any of the following occurs:

The draft reaches the planned endgame round

The user has two or fewer roster spots left

At least two K/DEF selections occurred among the most recent league-sized block of picks

The engine will not recommend a second kicker or defense after the user has filled the configured starter count at that position.

This is a decision-support score, not a probability model or optimizer. Its purpose is to combine value, need, and draft timing into a readable shortlist.

11. FastAPI service

The backend listens locally at `http://127.0.0.1:8765`.

Method	Endpoint	Purpose
GET	`/api/health`	Backend readiness check
GET	`/api/league`	Current effective league settings and draft slot
PUT	`/api/league`	Validate and persist league/roster settings
GET	`/api/draft`	Draft status, order, picks, next pick, and user roster
GET	`/api/players/available`	Undrafted player pool, optionally filtered
POST	`/api/draft/picks`	Record a selected player
DELETE	`/api/draft/picks/latest`	Undo the latest selection
POST	`/api/draft/reset`	Clear picks and return to pick 1
GET	`/api/draft/recommendations`	Return the current recommendation shortlist

The available-player endpoint returns all categories of player. It sorts offensive model-ranked players first, then ranked K/DEF, then remaining players with ADP, then fully unranked players alphabetically. The Java client requests the full 599-player limit and performs interactive search and position filtering locally.

FastAPI's interactive documentation is available at:

```
http://127.0.0.1:8765/docs
```

12. JavaFX desktop application

12.1 Role

JavaFX is the user-facing shell. It does not recalculate projections or VOR. It calls the Python API asynchronously and maps JSON into small Java records:

`PlayerRow` for the available-player table

`DraftPickRow` for rosters and recent picks

`RecommendationRow` for recommendation results

Jackson handles JSON serialization and parsing. Java's `HttpClient` is forced to HTTP/1.1 to avoid unsupported HTTP/2 upgrade behavior with Uvicorn.

12.2 Main interface

The main 1200×1000 window contains:

- Backend connection status
- Refresh button
- League Settings button
- Reset Draft button
- Current overall pick, round, and pick within round
- Team on the clock
- User's next pick

- Searchable complete available-player table
- Position filter for ALL/QB/RB/WR/TE/K/DEF
- Draft Selected Player button
- Undo Last Pick button
- Recommendation table
- Selectable roster view for every fantasy team
- Ten most recent picks

Available-player columns include model rank, name, position, NFL team, tier, projected points, VOR, ADP, and value label or REVIEW indicator.

Recommendation columns include player, position, NFL team, recommendation score, VOR, roster need, ADP, return likelihood, and review warning.

12.3 League settings dialog

The settings dialog validates whole-number roster settings and limits FLEX eligibility to RB/WR/TE. Saving calls `PUT /api/league`. If draft-order settings change without existing picks, the backend rebuilds the empty order while keeping the user's team name.

12.4 Draft safety

Reset Draft uses a confirmation dialog. Undo only removes the latest pick. Drafting requires a selected player, and the backend rejects already-drafted players.

12.5 Embedded backend lifecycle

`BackendService` expects the Java application to be launched from `java-app` and the Python virtual environment at the project root as `.venv/bin/python`.

On startup it:

- Runs the complete Python data pipeline synchronously in a background Java thread.

- Waits for the pipeline process to finish successfully.
- Starts Uvicorn on localhost port 8765.
- When Java exits, `stop()` destroys the child Uvicorn process.

This produces a mostly single-application experience, although Java, Python, the virtual environment, local data files, network access, and the manual FantasyPros ADP file are still runtime dependencies.

13. Installation and operation

Python environment

From the project root:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r python_engine/requirements.txt
```

Python dependencies are:

- Requests 2.x
- Beautiful Soup 4.x
- FastAPI 0.x
- Uvicorn 0.x

Despite the existing README mentioning pandas and NumPy, the supplied requirements and implementation do not use them.

Java environment

The Maven project targets Java 26, JavaFX 26.0.1, Jackson Databind 2.21.4, Maven Compiler Plugin 3.14.1, and JavaFX Maven Plugin 0.0.8.

Run from `java-app`:

```
mvn clean javafx:run
```

The JavaFX native-access option is already configured in `pom.xml`.

Manual backend operation

For development or troubleshooting, start FastAPI independently from the project root:

```
source .venv/bin/activate
python -m uvicorn python_engine.api.main:app \
  --host 127.0.0.1 --port 8765 --reload
```

If a separate backend is already bound to port 8765, the embedded Java backend cannot bind to the same port.

14. Data and state lifecycle

Refreshable analytical data

The Sleeper, CBS, cleaned ADP, merged, and value CSV files are reproducible outputs, assuming the external sources remain available in compatible formats and `datasets/adp.csv` is present.

Persistent user state

Two files should be treated as user state:

`python_engine/data/league_settings.json`

`python_engine/data/draft_state.json`

Resetting the draft clears picks but does not erase league settings. Rebuilding analytics does not intentionally clear draft state.

Backups

Before a real draft, it is prudent to copy `draft_state.json` periodically. It contains the complete draft history and can restore the board after an application failure. A copied ZIP of the whole project also preserves the exact rankings and source data used on draft night.

15. Error handling and diagnostics

The pipeline validates the presence of the manual ADP file and stops at the first failed stage. Cleaner and merger scripts print match totals that help identify source-format or identity-matching regressions.

The API uses:

- 400 for invalid draft actions
- 404 for missing state or data files
- 409 when league-size/draft-slot changes conflict with existing picks
- FastAPI/Pydantic validation responses for malformed request bodies

The Java client includes non-2xx response bodies in its internal exception, although the visible UI generally displays a shorter error message. For deeper diagnosis, the inherited Python/Java terminal output remains valuable.

Useful diagnostic commands include:

```
python -m python_engine.scripts.inspect_adp
python -m python_engine.scripts.inspect_players
python -m python_engine.scripts.inspect_projection_matches
python -m python_engine.scripts.inspect_unmatched_projections
```

16. Known limitations and technical risks

16.1 League settings can outpace analytics

Saving a new team count, starter structure, bench size, or FLEX eligibility updates runtime configuration and possibly draft order, but the API does **not** call `calculate_vor.py`. The existing `player_values.csv` therefore continues to reflect the prior settings until the pipeline or VOR calculation runs again.

This matters because league size and roster composition directly determine replacement levels, tiers, draft relevance, and recommendations. The safest current workflow after changing

material league settings is to restart the Java application, which reruns the pipeline, or manually run:

```
python -m python_engine.analytics.calculate_vor
```

16.2 Bench distribution is fixed

Changing the number of bench spots does not rescale `BENCH_DISTRIBUTION`. Its entries remain fixed at a sum of five. If bench size changes, expected rostered counts will not represent all configured bench spots unless those assumptions are manually updated.

16.3 Startup always refreshes external data

The Java app runs the complete pipeline before starting FastAPI. Consequently, app startup depends on internet access, Sleeper availability, CBS page compatibility, and a valid FantasyPros ADP file. This is risky immediately before or during a live draft. A network or scraping failure can prevent the backend from launching even when valid cached datasets already exist.

16.4 CBS scraping is fragile

CBS HTML structure and column order are external contracts the project does not control. Cleaner logic is position-specific. A source redesign may require code changes before projections can refresh.

16.5 Defense points are estimates

Season defense scoring based on average points allowed per game cannot reproduce weekly bracket outcomes. Defense projections should be treated as directional rankings rather than exact totals.

16.6 Some configured scoring categories are unused

Two-point conversions, blocked kicks, and several special-teams scoring fields are configured but cannot be calculated without corresponding projection statistics.

16.7 Recommendations are heuristic

The recommendation score does not model uncertainty, correlation, bye-week conflicts, stacking, schedule strength, playoff schedule, auction prices, opponent roster strategy, or probability distributions. It is a transparent heuristic based on projection, replacement value, roster need, ADP, and timing.

16.8 Potential duplicate startup prompt

`FantasyFootballManagerApp.start()` requests a reset confirmation after `BackendService.start()` completes, and also schedules another reset confirmation with a 1.5-second pause. Depending on timing, the startup reset prompt can appear twice or before the backend is ready.

16.9 Configuration snapshot inconsistency

The supplied `config.py` defaults to 10 teams, while persisted settings and draft state describe 8 teams. This is allowed by the override design, but it can confuse someone reading only the source defaults. Documentation and UI should always identify the effective persisted configuration.

16.10 Packaging is not yet standalone

The Java app behaves like one interface but is not yet a self-contained `.app`. It assumes a particular project directory structure, Maven/Java compatibility, `.venv/bin/python`, source files, CSV/JSON data, and localhost port 8765.

17. Recommended hardening roadmap

Highest priority

- **Separate "start app" from "refresh data."** Start FastAPI immediately from cached data. Make refresh an explicit button or background operation so draft-night startup never depends on the internet.

- **Recalculate values after relevant settings changes.** Team count, starter counts, bench size, and FLEX eligibility should trigger VOR regeneration before the UI accepts the new model as current.
- **Make bench assumptions editable or proportional.** Validate that bench-distribution values sum to the configured bench size.
- **Remove the duplicate startup reset trigger.** Use one readiness sequence: start backend, poll `/api/health`, then prompt once.
- **Add automatic state backups.** Save timestamped copies of `draft_state.json` after every pick or every few picks.

Medium priority

- Add a visible "data updated at" timestamp and source-health summary.
- Display projection-estimate and missing-projection indicators in the player table.
- Let recommendation rows draft a player directly, ideally with confirmation.
- Add roster-slot validation and warnings for impossible constructions.
- Add automated tests for snake order, VOR boundaries, FLEX logic, duplicate picks, undo, settings validation, and API responses.
- Replace commented-out code in the available-player endpoint with a clear permanent implementation.
- Improve Java error dialogs to show the backend's useful response detail.

Longer-term

- Package the app with `jpackage`, a bundled Java runtime, and a bundled Python environment or compiled backend.

- Add post-draft roster analysis, waiver-wire replacement values, weekly projections, lineup optimization, trade evaluation, and free-agent recommendations.
- Add projection blending from multiple sources and confidence/range estimates.
- Add opponent-needs modeling to improve return probabilities during a live draft.

18. Draft-night operating checklist

Before draft day

- Update `datasets/adp.csv` with the latest compatible FantasyPros export.
- Confirm league scoring, roster, team count, and draft slot.
- Run the pipeline while internet access is stable.
- Review match counts and any cleaner warnings.
- Confirm `player_values.csv` has a current modification time.
- Open `/api/health`, `/api/league`, and `/api/players/available` if diagnosing the backend.
- Start a test draft and verify snake order.
- Test draft, undo, reset, roster selection, filters, and recommendations.
- Save a backup ZIP after the successful refresh.

Immediately before the draft

- Avoid refreshing external data unless necessary.
- Confirm the Java header shows the correct team on the clock and your next pick.
- Confirm the draft order contains the correct number of teams and correct user slot.
- Keep the application terminal visible for unexpected backend errors.
- Keep the CLI recovery commands available.

During the draft

- Record every pick in order.
- Use Undo immediately if the wrong player is entered.
- Search the complete pool when an unranked or unprojected player is selected.
- Treat REVIEW flags as prompts to verify news and role, not automatic vetoes.
- Compare recommendation score, VOR, roster need, ADP, and return likelihood together.
- Do not over-trust defense point precision.

After the draft

Preserve `draft_state.json` as the historical record.

Export or copy the final roster information before resetting.

Use the roster data as the starting point for future lineup, waiver, and trade modules.

19. Conceptual interpretation

The strongest design choice in this system is the separation of three ideas:

- **Projection:** how many fantasy points the player is expected to score under this league's rules.
- **Replacement value:** how difficult those points are to replace at the player's position in this specific league format.
- **Acquisition cost:** where the market expects the player to be drafted.

Raw points answer, "Who scores the most?" VOR answers, "Who creates the largest advantage over an available alternative?" ADP comparison answers, "When must I pay for that advantage?" The recommendation engine adds the final draft-night context: "Do I need this position, and is the player likely to return?"

That layered approach is what turns the project from a projection viewer into a genuine decision-support system.

20. Final assessment

Fantasy Football Manager has reached a meaningful working prototype. Its data pipeline is reproducible, its custom scoring is transparent, its VOR model is league-aware, its draft state is persistent, its local API cleanly separates analytics from presentation, and its JavaFX interface supports the essential live-draft workflow.

The system is especially well suited to a user who wants to understand why a player is recommended rather than accept a black-box rank. VOR, ADP gaps, tiers, need labels, return likelihood, and review flags are all interpretable.

The main work remaining is operational hardening rather than proving the concept. Decoupling startup from data refresh, recalculating analytics after settings changes, adding tests and backups, and packaging the runtime would move the project from a strong personal prototype toward dependable standalone software. Its existing architecture also provides a practical foundation for a midseason manager: the draft-state roster model, data ingestion layer, scoring engine, and local API can be extended to weekly projections, lineup decisions, waiver claims, and trade analysis without discarding the current system.